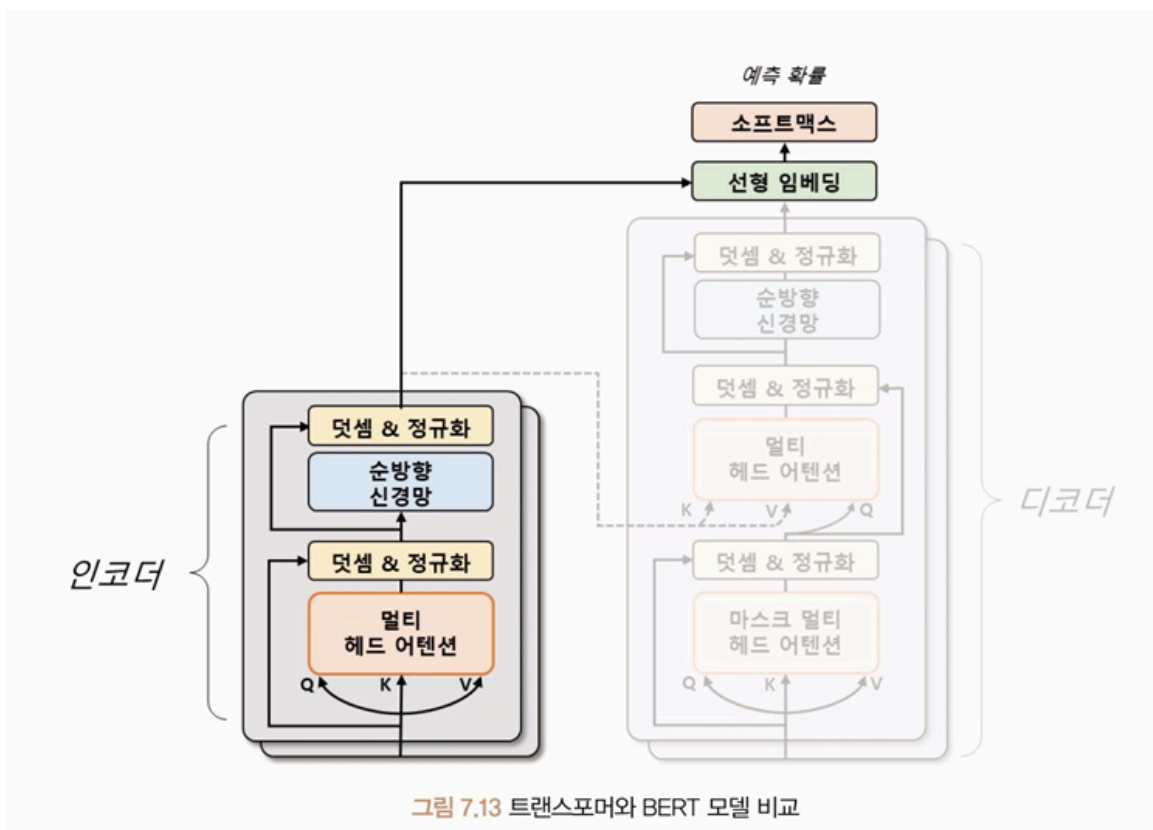


7장(BERT~)

425p~

BERT(Bidirectional Encoder Representations from Transformers)

- 2018년 구글 - 트랜스포머 기반 양방향 인코더 자연어 처리 모델
- 양방향 인코더 → 기존 모델들보다 문맥을 더 정확하게 파악하고, 다양한 자연어 처리 작업에서 높은 성능
- 대규모 데이터로 사전 학습된 모델 → 전이 학습에 주로 활용
- 트랜스포머의 인코더 모듈만을 사용해 입력 문장 처리



사전 학습 방법

- 마스크된 언어 모델링(Masked Language Modeling, MLM)
: 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식

- ex) 'I'm learning PyTorch' - **마스킹** → 'I'm [MASK] PyTorch' ⇒ BERT가 [MASK]에 들어갈 단어 예측
- 즉 누락된 단어를 추론하는 능력

- **다음 문장 예측(Next Sentence Prediction, NSP)**

: 두 문장이 주어졌을 때, 두번째 문장이 첫번째 문장의 다음에 오는 문장인지 여부 판단하는 작업

- 문장 간 관계 학습, 문장 간 의미적 유사성 파악

- BERT가 학습 및 추론 과정에서 활용하는 특수 토큰

- **[CLS]** 토큰: 문장 시작 토큰
 - 모델이 입력 문장이 어떤 유형의 문장인지 미리 파악할 수 있게 함
 - ex) 긍정/부정, 특정 객체 언급 유무
- **[SEP]** 토큰: 문장 구분 토큰
- **[MASK]** 토큰: 임의의 단어 마스킹하는 토큰

- ex 구조) **[CLS] 문장1 [SEP] 문장2 [MASK] [SEP]**



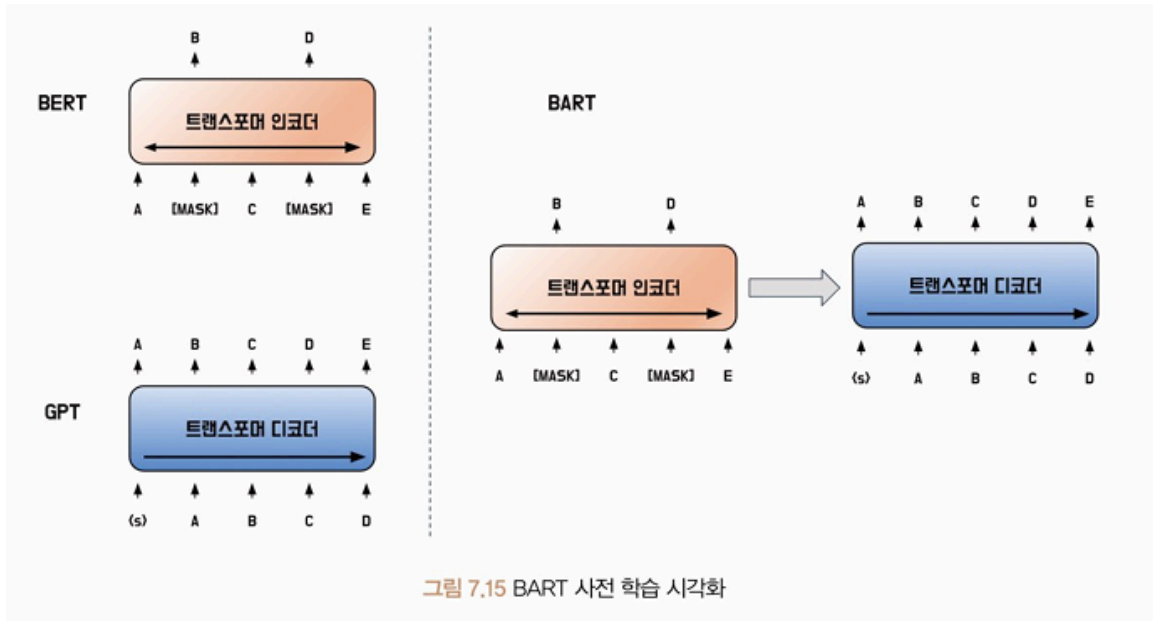
- 문장 시작에 [CLS] 토큰 추가
- 문장 마지막에 [SEP] 토큰 추가

- 토큰 중 15%에 해당하는 단어 대상으로 마스킹
 - 마스킹 대상 중 80%는 [MASK] 토큰으로 텍스트 단어 대체
 - 10%는 어휘 사전에 존재하는 무작위 단어로 변경
 - 10%는 실제 토큰 그대로 사용
- 사전 학습 → 미세 조정 기법 (계층 추가, 손실 함수 정의) ⇒ 자연어 처리 작업에 적용

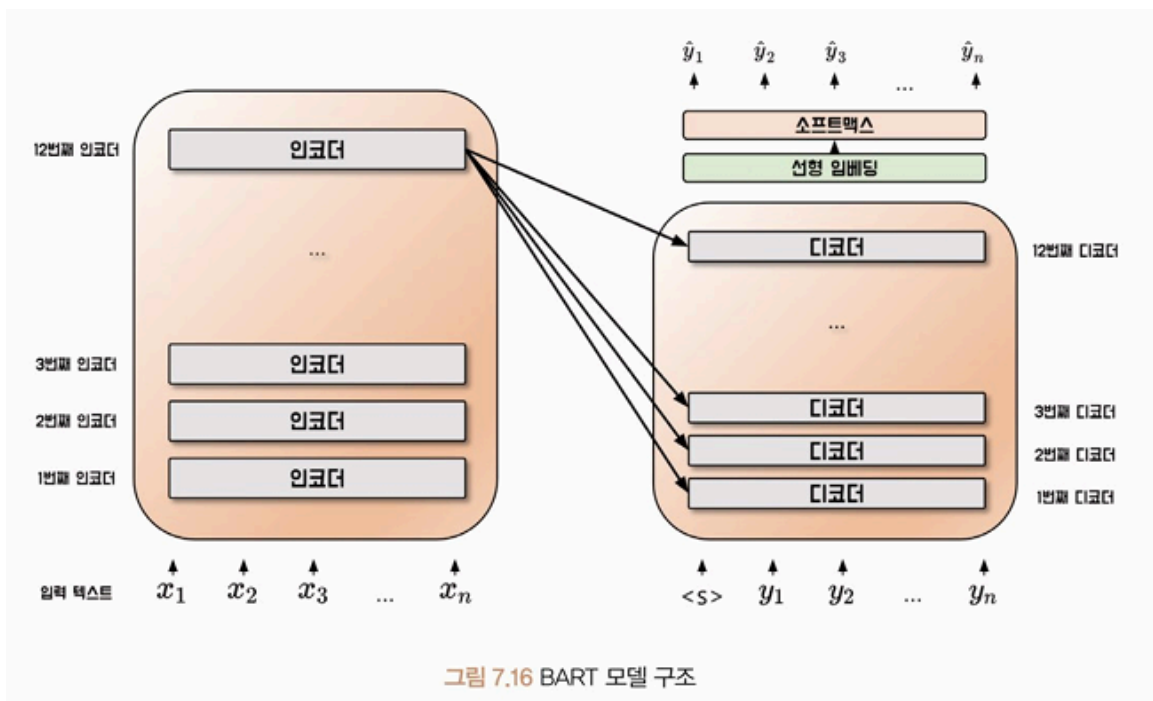
모델 실습

BART(Bidirectional Auto-Regressive Transformer)

- 2019년 메타 FAIR 연구소
- Seq2Seq (Sequence-to-Sequence) 구조
- 노이즈 제거 오토인코더 (Denoising Autoencoder) 로 사전 학습
: 입력 데이터에 잡음 추가 → 잡음 없는 원본 데이터로 복원하도록 학습
- BERT: 입력 문장 일부 단어를 무작위로 마스킹 → 마스킹된 단어를 맞추도록 학습
⇒ 문장 전체 맥락 이해, 단어 간 상호작용 파악
- GPT: 문장 이전 토큰들 입력 → 다음에 올 토큰 맞추도록 학습
⇒ 문장 내 단어들의 순서, 문맥 파악, 다음에 올 단어 예측
- BART는 BERT 인코더, GPT 디코더가 학습하는 방법을 일반화해 학습



- 노이즈가 추가된 텍스트 → 인코더 입력 (BERT 인코더의 마스킹처럼)
원본 텍스트 → 디코더 입력 ⇒ 디코더가 원본 텍스트를 생성할 수 있도록 학습
→ 문장 구조, 의미 보존하면서 다양한 변형 학습 가능
- BART 모델 구조



- 트랜스포머와의 차이점: BART는 인코더의 마지막 계층, 디코더의 각 계층 사이에만 어텐션 연산 수행 (트랜스포머는 모든 계층 사이의 어텐션 연산 수행)

- BART의 마지막 인코더 계층에서는 입력 문장 전체의 의미를 가장 잘 반영하는 벡터가 생성됨 → 디코더가 이 벡터를 출력 문장 생성할 때 참고함
- 이러한 방식의 장점: 정보 전달 최적화, 메모리 사용량 감소

사전 학습 방법

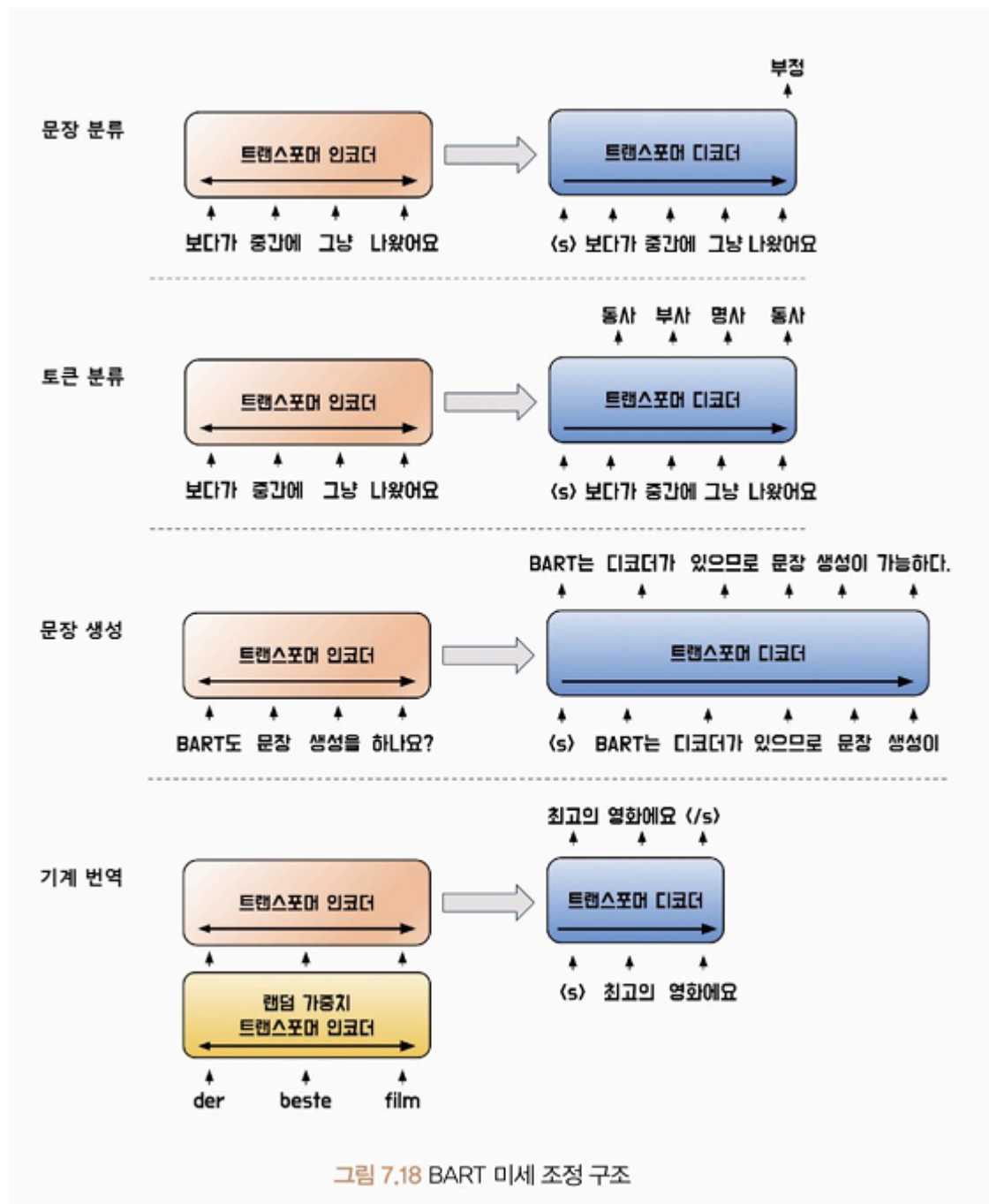


- **토큰 마스크(Token Masking)** : BERT의 MLM과 동일한 기법
- **토큰 삭제(Token Deletion)**: 문장 일부 토큰을 치환 대신 삭제함
 - 어떤 위치의 토큰이 삭제되었는지도 맞춰야 함
 - 입력 문장에서 불필요한 정보나 중요하지 않은 정보 자동 필터링해 처리
 - 모델 학습 및 예측 시간 감소, 모델의 일반화 성능 향상
- **문장 순열(Sentence Permutation)**: 마침표 기준으로 문장을 나누고 문장 순서를 섞는 방법
 - 원래 문장 순서를 맞히게 함
 - 문장 내에서 단어들이 어떻게 연결되어 있는지 잘 파악하게 함
 - 입력 문장이 다른 순서로 주어졌을 때 잘 처리하게 됨
- **문서 회전(Document Rotation)**: 임의의 토큰으로 문서 시작
 - 원래 시작 토큰 맞히게 함

- 모델이 문서의 시작점을 인식하게 함
- **텍스트 채우기(Text Infilling):** 몇 개 토큰을 하나의 구간(span)으로 묶고 일부 구간을 마스크 토큰으로 대체
 - 묶은 구간 길이(개수): 0 ~ 임의의 값(설정된 값)
 - 일반적으로 포아송 분포로 구간 나누고, 분포의 람다=3 으로 설정
 - 구간 길이=0 → 해당 위치에 변환된 토큰이 없다는 의미

미세 조정 방법

- 다운스트림 작업별로 인코더, 디코더에 다른 문장 구조를 입력해야 함



- 문장 분류 작업
 - 입력 문장을 인코더, 디코더에 동일하게 입력
 - 디코더 마지막 토큰 은닉 상태 → 선형 분류기 입력값
- 토큰 분류 작업
 - 입력 문장을 인코더, 디코더에 동일하게 입력
 - 디코더의 각 시점별 마지막 은닉 상태 → 토큰 분류기 입력값

- 문장 생성 작업 (추상적 질의응답(Abstractive Question Answering), 문장 요약(Summarization) 등)
 - 디코더를 통한 작업
- 기계 번역 작업
 - 사전 학습된 인코더 + 기계 번역을 위한 인코더 추가
 - 추가된 인코더는 기존 단어 사전을 사용하지 않아도 됨
 - 디코더는 사전 학습된 가중치와 단어 사전 사용
 - **학습 [2]**
 1. 새로 추가된 트랜스포머 인코더 가중치와 위치 임베딩, 첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치 학습
 2. 모든 신경망의 가중치를 작은 반복으로 학습

모델 실습

- 모델 평가 - 루지(Recall-Oriented Understudy for Gisting Evaluation, ROUGE) 점수 사용
 - 생성된 요약문과 정답 요약문이 얼마나 유사한지를 평가하기 위해 토큰의 N-gram 정밀도와 재현율을 이용해 평가하는 지표
 - ex) 유니그램 사용 → ROUGE-1 / 바이그램 사용 → ROUGE-2 / N-gram 사용 → ROUGE-N

정답 문장 유니그램

[대한민국, 은, 16, 강, 에, 진출, 했다]

생성된 문장 유니그램

[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]

ROUGE-1 재현율

$$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$$

ROUGE-1 정밀도

$$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$$

정답 문장 바이그램

[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]

생성된 문장 바이그램

[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]

ROUGE-2 재현율

$$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$$

그림 7.19 루지 점수 계산 방법

- ROUGE-L: 생성된 요약문과 정답 요약문 사이 **최장 공통부분 수열(Longest Common Subsequence, LCS)** 기반 통계 방식
 - 요약 문장이 입력 문장의 의미를 잘 전달하는지를 평가하는 데 유용
- ROUGE-LSUM: 개행 문자를 문장 경계로 인식해 각 문장 쌍의 LCS를 계산 → union-LCS 계산
 - union-LCS: 각 문장 쌍의 LCS를 합집합한 것. 중복된 부분을 제거한 후 길이를 계산한 것.
 - 요약의 정확성과 완전성을 모두 반영했는가의 지표
- ROUGE-W: 가중치가 적용된 LCS (Weighted LCS-based). 연속된 LCS 길이, 해당 부분 단어들에 가중치 부여해 평가.
 - 단어 간 유사도를 고려해 요약 문장이 입력 문장의 의미를 더 잘 전달하는지 평가하는 데 유용

rouge2 출력 결과

```
{
  'rouge1': 0.44116997177658945,
  'rouge2': 0.2,
  'rougeL': 0.4275670306001189,
  'rougeLsum': 0.4243807560903149
}
```

ELECTRA

(Efficiently Learning an Encoder that Classifiers Token Replacements Accurately)

- 2020년 구글
- 입력을 마스킹하는 대신 **생성자(Generator)**와 **판별자(Discriminator)** 사용해 사전 학습
 - 생성 모델: 실제 데이터와 비슷하게 토큰을 생성해 다른 토큰으로 대체
 - 판별 모델: 생성 모델이 만든 데이터와 실제 데이터를 입력받아 생성된 데이터와 실제 데이터 판별
- 생성적 적대 신경망(GAN)과 유사한 방법 → 더 효율적인 학습 가능
- 모델의 매개변수 수가 BERT에 비해 더 적음

사전 학습 방법

- 생성자 모델, 판별자 모델 모두 트랜스포머 인코더 구조를 따름
 - 생성자 모델: 입력 문장의 15% 토큰 마스크 처리 & 마스크 처리된 토큰이 원래 어떤 토큰이었는지 예측하며 학습
 - 판별자 모델: 각 입력 토큰이 원본 문장 토큰인지 생성자 모델로 인해 바뀐 토큰인지 판별하며 학습
 - RTD(Replaced Token Detection) 방식

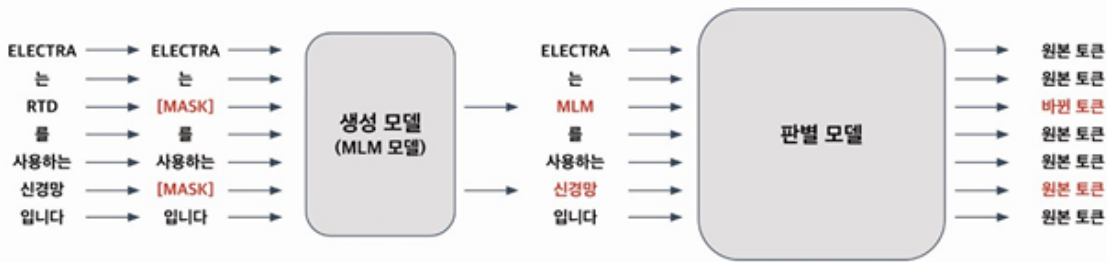


그림 7.20 ELECTRA 모델 학습 방법

- GAN과의 차이점
 - 생성자 모델이 예측한 토큰 == 원래 토큰일 경우
 - ELECTRA는 원본 토큰으로 간주
 - GAN은 원본이 아닌 생성된 토큰으로 간주
 - GAN은 생성 모델과 판별 모델을 **적대적**으로 학습함. 생성 모델은 판별 모델이 실제 데이터와 구분하기 어렵도록 학습하고, 판별 모델은 실제 데이터와 생성된 데이터를 더 잘 구분하게 학습함
 - GAN은 완전한 노이즈 벡터 입력받아 생성
- ELECTRA 생성 모델은 일부 토큰만 마스크 처리된 텍스트를 입력으로 받음

모델 실습

- 자연어 처리 분야에서 모델을 평가할 때 보편적으로 GLUE(General Language Understanding Evaluation) Benchmark 데이터셋 활용
- 문장 수준 또는 문서 수준의 이해력을 평가하는 데이터셋
 - 문장 분류, 문장 유사도 계산, 자연어 추론, 질의응답 등 11가지 과제로 모델 성능 평가

T5 (Text-to-Text Transfer Transformer)

- 2019년 구글
- 인코더-디코더 모델 구조
- GLUE, SuperGLUE, CNN/DM(Cable News Network/Daily Mail) 등 데이터셋에서 SOTA(State-of-the-art) 달성
- **입력과 출력을 모두 토큰(텍스트) 시퀀스로 처리하는 Text-to-Text 구조**
 - 입출력 형태를 자유롭게 다룰 수 있음

- 유연성, 확장성이 뛰어나 새로운 자연어 처리 작업에서도 쉽게 적용 가능
- 입출력 간 관계를 더욱 세밀하게 다룰 수 있음

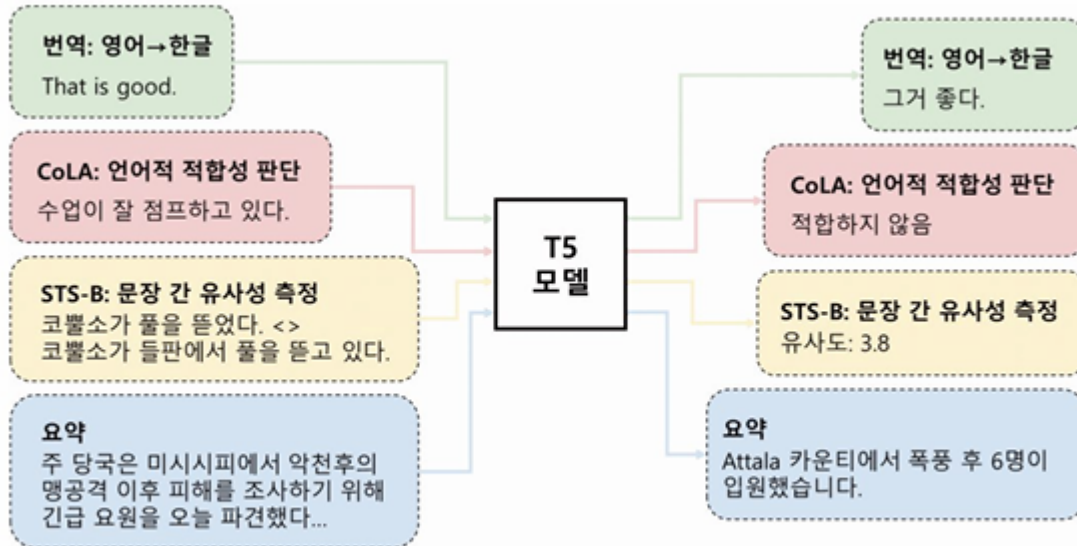


그림 7.21 T5 모델 학습 유형

- 사전 학습 후 미세 조정 단계에서 해당 작업 데이터를 이용해 모델 조정 → 최적의 성능 얻을 수 있음
 - CoLA 데이터셋 → 문법적으로 허용 가능한 문장/그렇지 않은 문장 구분 가능
 - STS-B(The Semantic Textual Similarity Benchmark) 데이터셋 → 문장 간의 의미적 유사성 측정 가능
 - 두 개의 문장 쌍이 주어졌을 때 그 문장 쌍의 의미적 유사성을 0~5까지의 점수로 평가하는 데이터세트
- 학습 데이터셋 = C4(Colossal Clean Crawled Corpus) 데이터셋 원본 문장, 대상 문장 사용
 - C4: 대규모 웹 크롤링 데이터셋
- 사전 학습 방식
 - 비지도 학습 방식
 - 입력 문장 일부 구간 마스킹해 입력 시퀀스 처리
 - 출력 시퀀스는 실제 마스킹된 토큰+마스크 토큰의 연결로 구성
 - 센티널 토큰(Sentinel Token) 사용
 - 문장마다 유일한 마스크 토큰

- '<extra_id_0>', '<extra_id_1>' 등 100개 기본값 사용
 - T5는 이런 마스킹 토큰을 예측하는 것을 목적으로 사전 학습됨
- 미세 조정
 - 지도 학습 방식
 - 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용

모델 실습